

JENKINS CHALLENGE

Create a Declarative Jenkins parameterized pipeline for simplecustomerapp.

Requirements:

- a. **Create a Pipeline job in Jenkins.**
 - b. **Use a Git repository for simplecustomerapp.**
 - c. **Add parameters:**
 - **BRANCH_NAME (string, default: main)**
 - **ENV (choice: dev, qa)**
 - a. **Define environment variables:**
 - **APP_NAME = simplecustomerapp**
 - a. **Implement stages:**
 - **Checkout (based on BRANCH_NAME)**
 - **Build (mvn clean compile)**
 - **Test (mvn test)**
 - **Package (mvn package)**
 - a. **Add post actions:**
 - **Success → print success message with environment**
 - **Failure → print failure message**
 - a. **Trigger using Build with Parameters.**
-
- a. **Create a Pipeline job in Jenkins.**
 - b. **GO to Jenkins dashboard and create a new job for our parameterized job where it should get multiple pipeline job, and have an option choose one**
 - c.
 - d. **I have clicked on add new and selected pipeline and clicked on OK**

JENKINS CHALLENGE

new Item

Search

New Item

Enter an item name

Jenkins_job

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**

OK

Selected parameter job option and selected branch name and description

Gmail YouTube News Maps Translate Google Data Science Mind...

Jenkins / Jenkins_job / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

☒ This project is parameterized ?

String Parameter ?

Name ?
BRANCH_NAME

Default Value ?
feature-1.1

Description ?
Git branch to build

Plain text [Preview](#)

☐ Trim the string ?

Choice Parameter ?

Save Apply

Provided following pipeline script along with the parameter

```
pipeline {  
    agent any
```

JENKINS CHALLENGE

```
parameters {
    string(
        name: 'BRANCH_NAME',
        defaultValue: 'feature-1.1',
        description: 'Git branch to build'
    )
    choice(
        name: 'ENV',
        choices: ['dev', 'qa'],
        description: 'Deployment environment'
    )
}

stages {

    stage('Checkout') {
        steps {
            echo "Checking out branch: ${params.BRANCH_NAME}"
            git branch: "${params.BRANCH_NAME}",
                url: 'https://github.com/betawins/sabear_simplecutomerapp.git'
        }
    }

    stage('Build') {
        steps {
            echo "Building application for ${params.ENV} environment..."
            sh 'mvn clean compile'
```

JENKINS CHALLENGE

```
    }  
  }  
  
  stage('Test') {  
    steps {  
      echo "Running tests for ${params.ENV} environment..."  
      sh 'mvn test'  
    }  
  }  
}  
  
post {  
  success {  
    slackSend message: "✅ SUCCESS | Job: ${env.JOB_NAME} | Branch:  
${params.BRANCH_NAME} | ENV: ${params.ENV}"  
  }  
  failure {  
    slackSend message: "❌ FAILED | Job: ${env.JOB_NAME} | Branch:  
${params.BRANCH_NAME} | ENV: ${params.ENV}"  
  }  
}  
}
```

Provided script in script shell

JENKINS CHALLENGE

- Use a Git repository for simplecustomerapp.
- Add parameters:

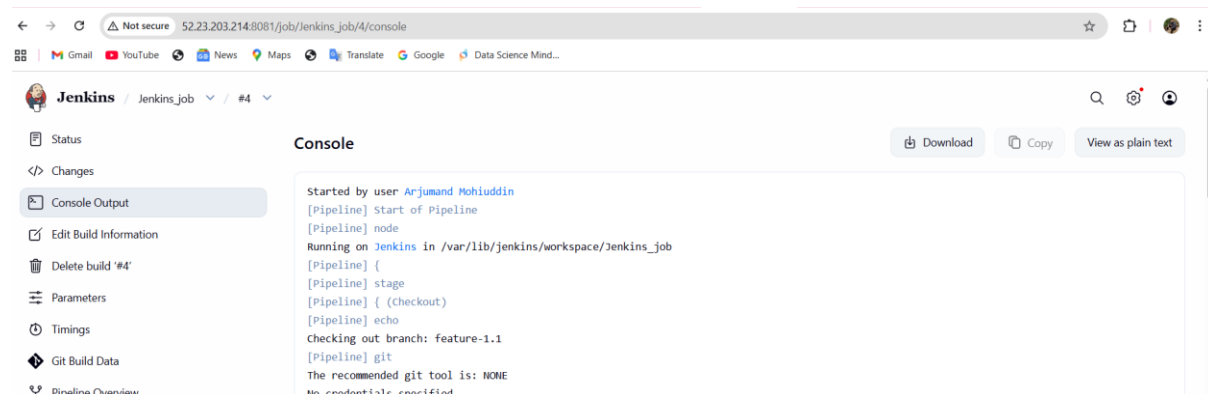
Added this cript in the below pipeline



And clicked to save job and build the job I got trigged and successfully executed

- Success → print success message with environment
- Failure → print failure message

Add git repo and select pipeline and job was executed



Here is the pipeline which I have executed



JENKINS CHALLENGE

Stages of the pipelines as shown below

The screenshot displays the Jenkins web interface for a pipeline named 'Jenkins_job'. The left sidebar shows a list of stages: Checkout (0.34s), Build (3s), Test (3s), and Post Actions (0.14s). The main panel shows the 'Git' stage, which is currently selected and has a duration of 0.27s. The stage's log output is visible, showing the execution of git commands to clone the repository and fetch changes. Below the 'Git' stage, the 'mvn clean compile' stage is shown with a duration of 2s. Its log output displays various Maven build messages, including warnings and information about the build process.

```
1 No credentials specified
2 Cloning the remote Git repository
3 Cloning repository https://github.com/betawins/sabear_simplecoterapp.git
4 > git init /var/lib/jenkins/workspace/Jenkins_job # timeout=10
5 Fetching upstream changes from https://github.com/betawins/sabear_simplecoterapp.git
6 > git --version # timeout=10
7 > git --version 'git version 2.50.1'
8 > git fetch --tags --force --progress -- https://github.com/betawins/sabear_simplecoterapp.git +refs/heads/*:refs/remotes/origin/* # timeout=10
```

```
9 [WARNING]
10 [INFO]
11 [INFO] -----< com.javatpoint:SimpleCustomerApp >-----
12 [INFO] Building SimpleCustomerApp 4-SNAPSHOT
13 [INFO] -----[ war ]-----
14 [INFO]
15 [INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ SimpleCustomerApp ---
16 [INFO]
17 [INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ SimpleCustomerApp ---
18 [WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
19 [INFO] skip non existing resourceDirectory /var/lib/jenkins/workspace/Jenkins_job/src/main/resources
20 [INFO]
21 [INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ SimpleCustomerApp ---
```

Post action also also we can see below

The screenshot shows the Jenkins web interface for the 'Jenkins_job' pipeline, specifically the '#4' stage. The left sidebar shows the stages: Checkout (0.34s), Build (3s), Test (3s), and Post Actions (0.14s). The main panel shows the 'Post Actions' stage, which is currently selected and has a duration of 0.14s. The stage's log output is visible, showing the execution of the 'Send Slack Message' action. The log output indicates that the action was successful and provides details about the message sent, including the job name, branch, and environment.

```
0 Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: techiehorizon, channel: jenkins-integration, color: <empty>, botuser: false, tokenCredentialId: slack-notification3, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
```